2026 青光眼分类项目代码与算法说明

更新日期：2026-10-14

固定协作方式：修改代码后只做语法、接口等轻量检查；训练和整批预测由用户自行运行。 train.py 只训练并使用验证集验证；只有单独运行 predict.py 才生成提交答案 CSV。

1. 项目目标与官方规则

本项目对彩色眼底图片进行二分类，输出图片属于青光眼阳性类别的概率。标签 1 表示 阳性，标签 0 表示阴性。2026 年赛项一提供训练集 51396 张、验证集 5849 张、 测试集 5855 张；图像为 JPG/PNG，尺寸不固定，阳性样本比例极低。

初赛不再上传代码等待服务器运行，而是在本地完成训练和预测，只上传 UTF-8 编码的 submission.csv。复赛需要同时提交预测结果和模型。

官方评分以 AUROC 为主要指标、AUPRC 为辅助指标。二者都依赖连续概率，因此 CSV 中的 Probability 必须是“阳性类别 1 的概率”，不能写成最大类别置信度。 平台显示的 50.00 是评分，不应直接解释成“50% 的图片分类正确”。本地准确率、 AUROC、AUPRC 和平台分数分别记录，不能互相替代。

2. 代码来源与修改边界

四步主体继续使用学习资料中的 day07 CNN 案例：

```
todo1 创建数据集 → model.py/create_dataset
todo2 搭建卷积网络 → model.py/ImageModel
todo3 模型训练    → train.py/train
todo4 模型测试    → predict.py/evaluate
```

卷积、ReLU、最大池化、拉平、全连接、交叉熵、Adam、梯度清零、反向传播和参数更新 仍沿用课堂逻辑。2026 规程带来的必要修改是：独立验证集、本地预测、固定结果路径、 官方 CSV 字段、阳性概率定义和类别均衡采样。本轮保留原 CNN 层次与参数形状，主要 修改采样、过程显示、每轮验证和最佳模型保存；提交预测使用独立运行入口。

ANN 和课程 Markdown 仍只提供增加深度、Dropout、标准化和 Adam 等优化依据，不替代 CNN 主体。

3. 项目结构与真实数据

```
submission/
├─ code/
│  ├─ model.py
│  ├─ train.py
│  ├─ predict.py
│  ├─ utils.py
│  └─ requirements.txt
├─ model/
│  ├─ best_model.pth
│  └─ config.json
└─ results/
   ├─ submission.csv
   └─ training_history.csv   # 本地训练记录，不上传

glaucoma_first/
├─ train.csv / train/      # 51396，CSV 列 Image_name/Age/True_Label
├─ valid.csv / valid/      # 5849
└─ test.csv / test/        # 5855
项目文档/
README.md
```

以下为上一版首轮基线的历史结果，不能当作本轮修改后的结果：训练 51396、验证 5849， 1 epoch，loss 0.49328，训练打印 acc 0.97，耗时 1624.53s。验证 accuracy 0.967345、AUROC 0.628867、 AP 0.048873；此前文档曾将 AP 与

AUPRC 并称，本版将二者分开。验证阳性概率范围 0.064293～0.308502。0.5 阈值混淆矩阵 TP=0、TN=5658、FP=0、FN=191，sensitivity=0、specificity=1。训练准确率约 97% 被 类别不平衡误导，不能代表阳性识别效果。该模型仅为首轮流程基线，不是最终竞赛方案。

每次运行默认继续训练 3 轮，batch size 为 64，CPU 线程数为 8。 2026-10-13 已完成的三轮训练，最佳轮为第 3 轮：验证 AUROC 0.885387、 梯形 AUPRC 0.362978、AP 0.365991、准确率 0.915199、阳性召回率 64.40%， 混淆矩阵 TP=123、TN=5230、FP=428、FN=68。以上是之前运行的记录， 本次职责分离修改没有重新训练；下一次训练由用户执行。

submission/code/ 是唯一 Python 源码目录。初赛实际上传时只选择 submission/results/submission.csv。

4. model.py

4.1 create_dataset()

来源：复用课堂 CNN 的 todo1。

作用：调用 utils.load_train_data()，返回 train_dataset 和 valid_dataset。 若已放入官方验证集，使用官方验证集；否则工具函数才进行临时 8:2 分层划分。

4.2 ImageModel.__init__()

来源：复用课堂 CNN 的网络定义，并参考课堂优化资料增加层数和参数量。

模型结构：

```
输入 RGB：3 × 64 × 64
→ Conv2d(3, 32, 3) → ReLU → MaxPool2d(2)       = 32 × 31 × 31
→ Conv2d(32, 64, 3) → ReLU → MaxPool2d(2)      = 64 × 14 × 14
→ Conv2d(64, 128, 3) → ReLU → MaxPool2d(2)     = 128 × 6 × 6
→ reshape                                      4608
→ Linear(4608, 2048) → ReLU → Dropout(0.5)
→ Linear(2048, 2048) → ReLU → Dropout(0.5)
→ Linear(2048, 2)                              两类 logits
```

总参数约 1373 万，float32 参数文件约 54.9MB，低于规程的 500MB 上限。

4.3 ImageModel.forward(x)

来源：复用课堂 CNN 的前向传播。

依次完成三次“卷积加权求和 → ReLU 激活 → 最大池化”，再拉平并通过两层带 Dropout 的全连接网络。输出层不执行 Softmax，因为训练使用的 CrossEntropyLoss 已包含 相应的概率计算。

4.4 main

直接运行 model.py 时，只调用 torchsummary.summary 打印每层形状和参数量，不 训练、不预测、不生成模型或 CSV。

5. train.py

5.1 train(train_dataset, valid_dataset)

来源：复用课堂 CNN 的 todo3。仍使用原前向传播、交叉熵、Adam 和反向传播循环； 新增均衡采样、进度输出、验证比较和最佳模型保存。

流程：

1. 固定 PyTorch 随机种子为 0，设置 CPU 线程数为 8；

2. 创建 batch size 为 64 的 DataLoader，使用均衡采样器，不再设置 shuffle=True；

3. 创建 ImageModel；若 RESUME=True 且已有 .pth，先加载并验证旧模型；

4. 将训练前模型的指标记为 epoch=0，作为本轮比较的起点；

5. 创建普通 CrossEntropyLoss，均衡采样后不再额外加权损失；

6. 创建 Adam，学习率 1e-4，默认继续训练 3 轮；

7. 对每批数据执行前向传播、损失计算、梯度清零、反向传播和参数更新；

8. 首批、每 25 批和末批立即打印进度；每轮结束打印平均损失、训练准确率和时间；

9. 每轮调用 predict.evaluate(valid_dataset, model) 验证当前模型；

10. 先比较验证 AUROC，相同时比较梯形 AUPRC，仅在改善时保存最佳 .pth；

11. 更新 training_history.csv 和 config.json，打印本次保留模型的验证结果。

每轮抽取 51396 次，因此 batch size 为 64 时共 804 批。终端输出保留课堂的 epoch/loss/acc/time 字段，同时显示当前批次、完成百分比和预计剩余时间。以下仅为 输出形状示意，尖括号是运行时数值，不是实测结果：

```
训练程序已启动, 正在加载PyTorch和数据读取模块...
开始训练 epoch: 1/3
epoch: 1/3, batch: 25/804 (3.1%), time:<秒>, remaining:<分钟>min, loss:<损失>, acc:<准确率>
epoch: 1, loss: <损失>, acc:<准确率>, time:<秒>s
开始验证: 5849张有标签图片
Acc: <准确率>, AUROC: <数值>, AUPRC(梯形): <数值>, AP: <数值>
阳性召回率: <百分比>, 特异度: <百分比>, TP:<数量> TN:<数量> FP:<数量> FN:<数量>
```

所有进度均使用 flush=True，不再等整轮完成才出现第一条信息。剩余时间按照已完成 批次的平均耗时估计，首次读取图片与后续命中缓存的速度不同，估计值会变化。

5.2 均衡采样与续训原理

训练集阴性 49739 张、阳性 1657 张。若类别 c 的数量为 n_c，该类每张图片的 抽样权重为：

```
sample_weight = 1 / n_c
```

两个类别的总抽样权重都为 1，因此每次抽到阳性、阴性的概率各约一半。这是有放回 随机采样，不保证每批严格各一半，也不保证一轮中每张原图都出现一次。阳性图片会被 重复抽到，因此只对训练数据使用随机水平翻转；验证集和测试集不做随机增强。

上一版使用 batch=8 和加权交叉熵，绝大多数批次没有阳性；加权损失默认按每批的权重 总和归一化，全阴性批次中类别权重会抵消。本版改用均衡采样与普通交叉熵，直接改变 每批接触到的样本分布。均衡采样后的训练 acc 与上一版自然分布上的 97% 不能直接 比较，应重点比较同一个官方验证集上的 AUROC、AP、召回率等指标。

RESUME=True 表示从已保存模型权重继续训练，但 Adam 的内部状态重新初始化， 不是恢复上次运行的完整中断点。将其改为 False 才会从随机初始化开始。每次运行的 轮次从 1 重新计数；best_epoch_this_run=0 表示训练后仍保留运行前的模型。

最佳模型保存时先写 best_model.tmp，完整写入后替换 best_model.pth。验证指标 不如原模型时不覆盖参数，最后一轮不一定是最终使用的模型。

5.3 main

直接运行 train.py 时：

```
打印启动信息 → create_dataset
→ train：可选验证旧参数 → 训练一轮 → 验证 → 保存最佳参数/训练记录（共3轮）
→ 打印训练结束提示并退出
```

train.py 结束时只保存模型、配置及训练历史，不读取 test/，也不生成或更新 提交答案 submission.csv。需要提交时由用户单独运行 predict.py。 每轮 Acc 来自有标签的 valid/，不是无标签 test/ 的真实准确率。

6. predict.py

6.1 evaluate(test_dataset, model=None)

来源：复用课堂 CNN 的 todo4。

由 train.py 每轮调用。传入 model 时直接验证当前参数；未传入时才从 best_model.pth 加载模型。函数切换 eval() 并使用 torch.no_grad() 关闭梯度， 按不打乱的顺序收集真实标签及 softmax(output)[:, 1]，调用 utils.calculate_metrics()，返回含

指标的字典，并打印验证进度和效果。

返回字段包括：

```
accuracy / auroc / auprc / ap / recall / specificity / tp / tn / fp / fn
```

其中 auprc 为 PR 曲线梯形积分，ap 为平均精度，计算定义不同，不能互称。 详细定义见第 7.8 节。验证不会更新模型权重。

6.2 predict(test_dir, output_csv)

来源：复用课堂测试循环，按 2026 规程改为本地无标签预测。

提交路径保持固定：

```
test_dir  = 项目根目录/glaucoma_first/test
output_csv = submission/results/submission.csv
```

流程：

1. 读取 test.csv，按官方 Image_name 顺序匹配 JPG/JPEG/PNG 图片；

2. 创建不打乱顺序的 DataLoader；

3. 加载 best_model.pth；

4. 切换 eval() 并关闭梯度；

5. 对 logits 执行 Softmax；

6. 只提取 softmax(output)[:, 1]，即阳性类别概率；

7. 调用工具写出官方 CSV；

8. 打印预测进度、生成路径、图片总数及预测为阳性的数量。

test.csv 只有 Image_name/Age，没有 True_Label，所以此阶段无法给出真实 准确率或 AUROC。需要用官方有标签 valid/ 观察模型效果，测试集最终成绩由平台给出。

6.3 main

直接运行 predict.py，不再需要服务器传入位置参数。main 只调用 predict()，最终 生成可直接上传的 submission.csv。train.py 只导入其中的 evaluate()； 导入模块不会触发预测或生成答案。

7. utils.py

工具函数均是为本项目的数据接口、训练管理和结果格式编写；Dataset 的标准方法 沿用 PyTorch 常规写法，具体来源也标在对应代码前。

7.1 ImageDataset.__init__(image_paths, labels=None, training=False)

保存图片路径和可选标签。有标签时用于训练/验证，无标签时用于最终预测。 training=True 选择随机水平翻转加标准化变换；其余情况使用确定性变换。同时创建 当前数据集对象的内存缓存，不向数据目录写入额外图片。

7.2 ImageDataset.__len__()

返回数据集样本数量，供 PyTorch DataLoader 计算批次数。

7.3 ImageDataset.__getitem__(index)

第一次访问某张图片时，先请求 JPEG 解码器按 64×64 附近尺寸读取，转换为 RGB， 以双线性插值缩放为 64×64 并缓存在内存。再次访问直接使用缓存，避免重复解码大图。 缓存保存的是未随机增强的 RGB 图片，每次训练取样仍重新决定是否水平翻转，然后执行 Resize → ToTensor → Normalize。有标签时返回 (x, y)，无标签时只返回 x。

训练和验证共 57245 张，若全部进入缓存，按每像素 3 字节估算的 RGB 像素数据约 671 MiB；实际进程内存还包括图像对象、索引、模型、梯度和优化器。缓存按需填充， 不预先复制完整数据集到磁盘；程序退出后释放。验证/测试没有随机翻转。

7.4 _load_labeled_images(csv_path, image_dir)

读取一份有标签 CSV，检查 Image_name/True_Label 字段、图片是否存在以及标签是否为 0/1， 返回图片路径和标签。该内部函数避免训练集与验证集读取代码重复。 CSV 中 .jpg 与真实文件的 .JPG 通过忽略大小写匹配，兼容大小写敏感文件系统。

7.5 load_train_data()

读取 glaucoma_first/train.csv 和 glaucoma_first/valid.csv，分别匹配 train/ 和 valid/ 图片目录，分别构建启用训练变换和确定性验证变换的数据集。只有缺少官方 验证 CSV 时，才按类别以固定随机状态做 8:2 分层切分。

7.6 create_sampler(labels)

统计两个类别的数量，给每张样本设置其类别数量的倒数作为权重，返回 WeightedRandomSampler。每轮抽取次数等于训练集原始数量，允许重复抽取。 训练数据缺少任一类别时立即报错。本函数替代旧的 calculate_class_weight()， 不会再给 CrossEntropyLoss 传类别权重。

7.7 print_progress(stage, batch_idx, batch_count, start, detail="")

训练、验证和预测共用的进度输出。第 1 批、每 25 批及最后一批打印阶段名、批次、 完成百分比、已用时间和预计剩余分钟数；训练另外传入累计平均损失和准确率。 LOG_EVERY 控制打印间隔，flush=True 让终端即时显示。

7.8 calculate_metrics(labels, positive_probability)

检查验证标签包含 0/1 两类且概率为有限值，使用 scikit-learn 计算：

字段	定义或作用
accuracy	(TP+TN)/总数，即普通分类准确率
auroc	roc_auc_score，根据连续阳性概率评估类别排序
auprc	先生成 precision-recall 曲线，再以 auc(recall, precision) 做梯形积分
ap	average_precision_score，按召回率增量加权的平均精度，不是梯形积分
recall	TP/(TP+FN)，阳性召回率，也称 sensitivity
specificity	TN/(TN+FP)，阴性正确识别比例
tp / tn / fp / fn	以概率 0.5 为分类阈值的混淆矩阵数量

AUROC、梯形 AUPRC 和 AP 使用连续概率；准确率、召回率和特异度使用 0.5 阈值。 本地同时列出两种 PR 汇总值，避免把历史 AP 误称为梯形 AUPRC；平台最终以其官方 计算实现为准。对阳性稀少数据，不能只用 Accuracy 判断训练是否有效。

7.9 save_training_record(history, best_metrics, best_epoch)

每轮写入 submission/results/training_history.csv，记录本次运行已完成各轮的 训练损失、训练准确率及验证指标。加载旧参数时还包含 epoch=0 的验证记录，该行 没有训练损失或训练准确率。

同时更新 submission/model/config.json，保存图片尺寸、batch size、本次计划 轮数、续训开关、采样方式、最佳轮次和最佳验证指标等。历史表每次运行记录本次过程， 不自动追加前次运行；best_epoch_this_run=0 明确表示旧参数仍最佳。这两个文件 用于本地查看，不上传初赛平台。

7.10 load_test_data(test_dir, test_csv)

读取官方 test.csv，按其中的 Image_name 顺序匹配测试目录中的 JPG/JPEG/PNG， 构建无标签 ImageDataset；匹配时忽略 .jpg/.JPG 大小写差异，写回 CSV 时保留官方名称。

7.11 write_submission(image_names, positive_probability, output_csv)

写出 UTF-8 CSV。字段严格为：

```
Image_name, Probability, Pred_Label
```

检查文件名数量与概率数量一致，概率有限且位于 [0,1]，再写文件。阳性概率先保留 6 位小数，再按是否大于等于 0.5 得到 Pred_Label，避免显示 0.500000 却写标签 0 的精度边界问题。函数返回实际输出路径。

8. 17 个函数和方法汇总

文件	函数/方法	作用
model.py	create_dataset	返回训练集和验证集
model.py	ImageModel.__init__	定义三卷积二分类网络
model.py	ImageModel.forward	定义前向传播
train.py	train	均衡训练、每轮验证并保存最佳参数
predict.py	evaluate	有标签验证并输出完整指标
predict.py	predict	本地预测并生成官方 CSV
utils.py	ImageDataset.__init__	保存路径、标签、变换和缓存
utils.py	ImageDataset.__len__	返回样本数
utils.py	ImageDataset.__getitem__	按需读取/缓存图片并变换
utils.py	_load_labeled_images	读取有标签 CSV 和图片
utils.py	load_train_data	读取训练/验证数据
utils.py	create_sampler	创建类别均衡采样器
utils.py	print_progress	显示训练、验证和预测进度
utils.py	calculate_metrics	计算AUROC、PR指标及混淆矩阵
utils.py	save_training_record	保存本次训练历史和最佳参数说明
utils.py	load_test_data	读取无标签测试图片
utils.py	write_submission	写出可提交 CSV

9. 提交 CSV 的算法

对于模型输出 z_0,z_1，阳性概率为：

```
Probability = exp(z_1) / (exp(z_0) + exp(z_1))
```

标签为：

```
Pred_Label = 1, Probability >= 0.5
Pred_Label = 0, Probability < 0.5
```

示例：若阴性概率为 0.88、阳性概率为 0.12，CSV 必须写 Probability=0.12 和 Pred_Label=0，不能写最大置信度 0.88。

10. 数据放置和运行

```
glaucoma_first/train.csv
glaucoma_first/train/*
glaucoma_first/valid.csv
glaucoma_first/valid/*
glaucoma_first/test.csv
glaucoma_first/test/*
```

本地训练：

```
& "C:\Users\LENOVO\.conda\envs\jitiaosai\python.exe" submission/code/train.py
```

在编辑器中直接运行 train.py 也会执行同一流程。启动后先显示模块加载和数据读取 信息；若已有参数，先验证旧模型；每轮训练后验证；全部训练结束后退出。 结果保存在固定路径，不随编辑器的工作目录变化。

本地预测：

```
& "C:\Users\LENOVO\.conda\envs\jitiaosai\python.exe" submission/code/predict.py
```

单独运行预测适用于已有模型、只需重新生成 CSV 的情况，不会重新训练。参数在 utils.py 中集中设置：EPOCHS=3、BATCH_SIZE=64、CPU_THREADS=8、 RESUME=True、LOG_EVERY=25。需要从头训练时可将 RESUME 改为 False； 观察效果应看 valid/ 指标，不能把没有标签的 test/ 当作本地计分集。

初赛上传：

```
submission/results/submission.csv
```

11. 环境与限制

规程推荐 Python 3.12、PyTorch 2.10.0、torchvision 0.25.0、CUDA 12.8 和 cuDNN 9，服务器参考硬件为 NVIDIA T4 16GB、8 核 CPU、32GB 内存。

本机 jitiaosai 当前为 Python 3.12.13、PyTorch 2.13.0+cpu、torchvision 0.28.0+cpu。requirements.txt 已按规程写入推荐框架版本，但没有自动降级现有环境， 也没有修改 deepStudy。 指标计算新增使用 scikit-learn，已列入 requirements.txt。本次代码仍以 CPU 运行；模型层次和 64×64 输入保持不变。内存缓存提升多轮读取速度，但实际峰值内存 会高于 671 MiB 的 RGB 像素估算值。

12. 当前状态和后续边界

训练、验证 CSV 列名为 Image_name/Age/True_Label，测试 CSV 只有 Image_name/Age。 以下阈值仅是上一版首轮模型的历史诊断，不适用于本轮新参数：F1 阈值 0.257520，F1 0.094390、 sensitivity 0.277487、specificity 0.844645；Youden 阈值 0.190134， sensitivity 0.790576、specificity 0.446271。上一版生成的提交 CSV 有 5855 个数据行， 列和测试图片名/顺序匹配；0.5 阈值下 Pred_Label 全为 0，但连续 Probability 可用于 AUROC。历史模型文件大小为 54935871 bytes。

目前代码具备进度显示、每轮验证、均衡采样、内存缓存和最佳参数保留。 训练与提交预测分开运行：train.py 完成训练和验证后退出；用户单独运行 predict.py 才生成或更新提交 CSV。助手修改后只做轻量检查，不自动启动训练或整批预测。 实测记录以 training_history.csv 和 config.json 为准。最终应比较同一官方验证集的 AUROC、梯形 AUPRC、AP 和阳性召回率，并在正式上传后记录真实平台分数。